TUTORIAL PASSO A PASSO · PYTHON PARA ANÁLISE DE DADOS

Analisando dados de eleições com Python

Do zero ao resultado: instalar, ler um CSV, calcular e mostrar tabelas no terminal, com pandas e rich.

O que você vai aprender

Instalar o **Python**, o **VS Code** e as bibliotecas **pandas** e **rich**.

Criar um **ambiente virtual** (venv) para o projeto.

Ler um arquivo **CSV** com dados de uma eleição fictícia.

Somar, agrupar, filtrar e ordenar dados com o **pandas**.

Mostrar os resultados em **tabelas no terminal** e salvar um novo CSV.

Parte	Conteúdo
1	Preparando o computador: Python, VS Code, venv e bibliotecas
2	Os dados: o arquivo `eleicoes.csv`
3	Arquivo de apoio: `formatacao.py`
4	As 6 etapas do programa, uma por arquivo
5	Juntando tudo no `main.py`
6	Problemas comuns e como resolver
7	Resumo dos comandos

Observação: todos os nomes, partidos e votos deste material são **fictícios**, criados apenas para fins didáticos.

O que vamos construir

Um programa que lê os votos de uma eleição e mostra quem ganhou em cada cidade.

Vamos partir de um arquivo CSV com os votos de cada candidato em cada seção eleitoral. O programa vai ler esse arquivo, calcular os totais e mostrar tudo em tabelas organizadas no terminal. No final, ele também salva um novo CSV com os prefeitos eleitos, pronto para abrir no Excel.

O programa é dividido em **etapas**, e cada etapa fica em um arquivo separado. Assim, podemos construir e testar uma etapa de cada vez, sem depender das próximas.

Resultado final

Quando terminarmos, o comando `python main.py` vai mostrar isto:

RESULTADO DE PYTHON MAIN.PY

Eleições 2024 (dados fictícios)

Registros: 32 Votos: 9.100 Cidades: 5 Candidatos: 16

Primeiras 5 linhas

estado	cidade	secao	cargo	candidato	partido	votos
SP	Campinas	0001	Prefeito	Ana Ribeiro	PFA	412
SP	Campinas	0001	Prefeito	Bruno Tavares	PNB	388
SP	Campinas	0001	Prefeito	Carla Mendes	PVC	157
SP	Campinas	0002	Prefeito	Ana Ribeiro	PFA	365
SP	Campinas	0002	Prefeito	Bruno Tavares	PNB	401

Votos por partido

Partido	Votos	%	Gráfico
PFA	3.603	39,6	
PVC	2.915	32,0	
PNB	2.582	28,4	

Prefeitos eleitos

UF	Cidade	Candidato	Partido	Votos	%
SP	Campinas	Bruno Tavares	PNB	789	41,6
PE	Olinda	Iara Santos	PNB	568	51,1
PE	Recife	Fabio Lins	PVC	1.008	41,9
SP	Santos	Elisa Moura	PNB	668	52,1
MG	Uberlandia	Karina Alves	PFA	843	50,6

Resultado salvo em: dados/resultado_prefeitos.csv

Parte 1 · Preparando o computador

Faça esta parte uma única vez em cada computador.

O que instalar	Para que serve	Onde conseguir
Python 3.13	A linguagem que vamos usar	python.org/downloads
VS Code	Editor onde escrevemos e rodamos o código	code.visualstudio.com
Extensão **Python** (Microsoft)	Faz o VS Code entender Python e mostra o botão de executar	Dentro do VS Code, aba Extensões
Biblioteca **pandas**	Ler e analisar tabelas (o CSV)	Comando `pip` (passo 1.5)
Biblioteca **rich**	Mostrar tabelas e cores no terminal	Comando `pip` (passo 1.5)

1.1 Instalar o Python

1. Acesse **python.org/downloads** e clique em **Download Python 3.13**.
2. Abra o instalador. **Antes de clicar em instalar**, marque a opção **Add python.exe to PATH** na parte de baixo da janela.
3. Clique em **Install Now** e aguarde terminar.

> **Atenção**
> Se você esquecer de marcar **Add python.exe to PATH**, o comando `python` não vai funcionar no terminal. Nesse caso, rode o instalador de novo, escolha **Modify** e marque a opção.

Também é possível instalar pelo terminal, sem abrir o navegador:

DIGITE NO TERMINAL (POWERSHELL)

```
winget install --id Python.Python.3.13 -e
```

Para conferir se deu certo, **abra um terminal novo** e digite:

DIGITE NO TERMINAL (POWERSHELL)

```
python --version
```

RESULTADO ESPERADO

```
Python 3.13.15
```

> **Dica**
> Se aparecer `Python was not found` ou abrir a Microsoft Store, feche o terminal e abra outro. O terminal que já estava aberto antes da instalação não enxerga o Python novo.

1.2 Instalar o VS Code e a extensão Python

1. Acesse **code.visualstudio.com**, baixe e instale o VS Code.
2. Abra o VS Code e clique no ícone de **Extensões** na barra lateral (ou `Ctrl+Shift+X`).
3. Pesquise por **Python** e instale a extensão publicada pela **Microsoft**.

1.3 Criar e abrir a pasta do projeto

1. Crie uma pasta chamada `eleicoes` (por exemplo, na Área de Trabalho).
2. No VS Code, vá em **Arquivo** → **Abrir Pasta...** e escolha a pasta `eleicoes`.
3. Abra o terminal do VS Code em **Terminal** → **Novo Terminal** (ou `Ctrl+'`).

> **Atenção**
>
> Abra sempre a **pasta** do projeto, e não um arquivo solto. O programa procura os dados em `dados/eleicoes.csv` a partir da pasta aberta. Se outra pasta estiver aberta, aparece o erro `FileNotFoundError`.

1.4 Criar o ambiente virtual (venv)

> **O que é um ambiente virtual?**
>
> O **ambiente virtual** é uma "caixa" separada onde ficam as bibliotecas **deste** projeto. Assim, cada projeto tem as suas bibliotecas e um não atrapalha o outro. Ele fica na pasta `.venv`, que o próprio Python cria.

No terminal do VS Code, dentro da pasta do projeto, crie o ambiente:

DIGITE NO TERMINAL (POWERSHELL)

```
python -m venv .venv
```

Depois, **ative** o ambiente:

DIGITE NO TERMINAL (POWERSHELL)

```
.\.venv\Scripts\Activate.ps1
```

Se deu certo, o começo da linha do terminal passa a mostrar `(.venv)`:

RESULTADO ESPERADO

```
(.venv) PS C:\Users\aluno\Desktop\eleicoes>
```

> **Atenção**
>
> Se aparecer o erro **"a execução de scripts foi desabilitada neste sistema"**, rode o comando abaixo uma única vez e depois ative o ambiente de novo:
>
> `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

Por último, avise o VS Code para usar esse ambiente: aperte `Ctrl+Shift+P`, digite **Python: Select Interpreter** e escolha a opção que tem `.venv`. Com isso, o botão **Executar** (triângulo no canto superior direito) passa a usar as bibliotecas do projeto.

1.5 Instalar as bibliotecas

Crie na pasta do projeto um arquivo chamado `requirements.txt` com a lista de bibliotecas:

ARQUIVO: REQUIREMENTS.TXT

```
pandas
rich
```

Com o `(.venv)` aparecendo no terminal, instale tudo de uma vez:

DIGITE NO TERMINAL (POWERSHELL)

```
pip install -r requirements.txt
```

Para conferir, rode:

DIGITE NO TERMINAL (POWERSHELL)

```
python -c "import pandas, rich; print('Tudo certo!')"
```

RESULTADO ESPERADO

```
Tudo certo!
```

> **Dica**
>
> O `requirements.txt` é útil porque qualquer pessoa consegue instalar as mesmas bibliotecas com um único comando. Também daria para instalar uma por uma: `pip install pandas rich`.

Parte 2 · Os dados

O arquivo CSV com os votos da eleição fictícia.

> **O que é um CSV?**
>
> **CSV** (valores separados por vírgula) é um arquivo de texto que guarda uma tabela. A primeira linha tem os **nomes das colunas**, e cada linha seguinte é um **registro**. Os valores de cada coluna são separados por vírgula. O Excel e o pandas conseguem abrir esse formato.

Crie uma pasta chamada `dados` dentro do projeto e, dentro dela, o arquivo `eleicoes.csv` com o conteúdo abaixo:

ARQUIVO: DADOS/ELEICOES.CSV

```
ano,estado,cidade,zona,secao,cargo,candidato,partido,numero,votos
2024,SP,Campinas,101,0001,Prefeito,Ana Ribeiro,PFA,11,412
2024,SP,Campinas,101,0001,Prefeito,Bruno Tavares,PNB,22,388
2024,SP,Campinas,101,0001,Prefeito,Carla Mendes,PVC,33,157
2024,SP,Campinas,101,0002,Prefeito,Ana Ribeiro,PFA,11,365
2024,SP,Campinas,101,0002,Prefeito,Bruno Tavares,PNB,22,401
2024,SP,Campinas,101,0002,Prefeito,Carla Mendes,PVC,33,172
2024,SP,Santos,118,0001,Prefeito,Diego Farias,PFA,11,298
2024,SP,Santos,118,0001,Prefeito,Elisa Moura,PNB,22,341
2024,SP,Santos,118,0002,Prefeito,Diego Farias,PFA,11,315
2024,SP,Santos,118,0002,Prefeito,Elisa Moura,PNB,22,327
2024,PE,Recife,005,0001,Prefeito,Fabio Lins,PVC,33,520
2024,PE,Recife,005,0001,Prefeito,Gabriela Costa,PFA,11,476
2024,PE,Recife,005,0001,Prefeito,Henrique Paz,PNB,22,203
2024,PE,Recife,005,0002,Prefeito,Fabio Lins,PVC,33,488
2024,PE,Recife,005,0002,Prefeito,Gabriela Costa,PFA,11,502
2024,PE,Recife,005,0002,Prefeito,Henrique Paz,PNB,22,219
2024,PE,Olinda,012,0001,Prefeito,Iara Santos,PNB,22,267
2024,PE,Olinda,012,0001,Prefeito,Joao Batista,PVC,33,289
2024,PE,Olinda,012,0002,Prefeito,Iara Santos,PNB,22,301
2024,PE,Olinda,012,0002,Prefeito,Joao Batista,PVC,33,254
2024,MG,Uberlandia,277,0001,Prefeito,Karina Alves,PFA,11,433
2024,MG,Uberlandia,277,0001,Prefeito,Lucas Prado,PVC,33,398
2024,MG,Uberlandia,277,0002,Prefeito,Karina Alves,PFA,11,410
2024,MG,Uberlandia,277,0002,Prefeito,Lucas Prado,PVC,33,425
2024,SP,Campinas,101,0001,Vereador,Marcos Lima,PFA,11123,87
2024,SP,Campinas,101,0001,Vereador,Nadia Rocha,PNB,22456,64
2024,SP,Campinas,101,0002,Vereador,Marcos Lima,PFA,11123,92
2024,SP,Campinas,101,0002,Vereador,Nadia Rocha,PNB,22456,71
2024,PE,Recife,005,0001,Vereador,Otavio Reis,PVC,33789,110
2024,PE,Recife,005,0001,Vereador,Paula Nunes,PFA,11321,95
2024,PE,Recife,005,0002,Vereador,Otavio Reis,PVC,33789,102
2024,PE,Recife,005,0002,Vereador,Paula Nunes,PFA,11321,118
```

> **Dica**
>
> Em aula, é mais rápido o professor distribuir o arquivo `eleicoes.csv` pronto do que todos digitarem as 33 linhas.

O que significa cada coluna

Coluna	Significado	Exemplo
`ano`	Ano da eleição	2024
`estado`	Sigla do estado (UF)	SP
`cidade`	Nome da cidade	Campinas
`zona`	Zona eleitoral	101
`secao`	Seção eleitoral (onde fica a urna)	0001
`cargo`	Cargo disputado	Prefeito
`candidato`	Nome do candidato	Ana Ribeiro
`partido`	Sigla do partido	PFA
`numero`	Número do candidato na urna	11
`votos`	Votos que o candidato teve **naquela seção**	412

Repare que cada candidato aparece em **várias linhas**, uma por seção. Para saber o total de votos de um candidato, vamos precisar **somar** essas linhas. Isso é feito nas etapas 4 e 5.

Parte 3 · Arquivo de apoio

Funções de exibição usadas por todas as etapas.

Antes das etapas, criamos o arquivo `formatacao.py`. Ele guarda duas coisas que várias etapas usam. Escrevendo uma vez aqui, não precisamos repetir em cada arquivo.

ARQUIVO: FORMATACAO.PY

```
"""Ferramentas de exibição usadas por todas as etapas."""

from rich.console import Console

# O "console" do rich é o que imprime tabelas e cores no terminal.
# Criamos um só aqui e todas as etapas usam o mesmo.
console = Console()


def numero_br(valor, casas=0):
    """Escreve um número no padrão brasileiro. Ex.: 1008 -> "1.008" e 41.6 -> "41,6"."""
    # O Python escreve no padrão americano: 1008.5 vira "1,008.5"
    texto = f"{valor:,.{casas}f}"

    # Precisamos trocar "," por "." e "." por ",".
    # Usamos um "X" temporário para as trocas não se misturarem.
    texto = texto.replace(",", "X")
    texto = texto.replace(".", ",")
    texto = texto.replace("X", ".")
    return texto
```

Entendendo o código

- `from rich.console import Console` traz o `Console` da biblioteca rich. É ele que desenha tabelas e cores no terminal.
- `console = Console()` cria um console que as outras etapas vão importar e usar.
- A função `numero_br` escreve números no **padrão brasileiro**: ponto para separar milhares e vírgula para decimais.
- Em `f"{valor:,.{casas}f}"`, a vírgula pede separador de milhar e `.1f` pede uma casa decimal. O Python usa o padrão americano (`1,008.5`), por isso fazemos as trocas com `replace`.

> **Dica**
>
> Este arquivo não mostra nada sozinho. Se você rodar `python formatacao.py`, nada acontece, e está certo: ele só guarda funções para os outros arquivos.

Parte 4 · As etapas

Cada etapa é um arquivo que funciona sozinho. Construa, rode e confira antes de passar para a próxima.

Como as etapas funcionam

Toda etapa termina com o bloco `if __name__ == "__main__":`. É ele que permite rodar a etapa sozinha para testar, com `python nome_do_arquivo.py` ou com o botão **Executar** do VS Code.

Etapa 1 · Carregar os dados

Objetivo: Ler o arquivo CSV e transformar em uma tabela que o Python entende (um DataFrame).

Crie o arquivo `etapa1_carregar.py` na pasta do projeto:

ARQUIVO: ETAPA1_CARREGAR.PY

```
"""Etapa 1: ler o arquivo CSV com os dados da eleição."""

import pandas as pd

# Caminho do arquivo, a partir da pasta do projeto
CAMINHO_CSV = "dados/eleicoes.csv"


def carregar_dados():
    # read_csv lê o arquivo e devolve uma tabela (DataFrame).
    # "zona" e "secao" são lidas como texto (str) para não perder
    # os zeros à esquerda: "005" continuaria "005", e não 5.
    df = pd.read_csv(CAMINHO_CSV, dtype={"zona": str, "secao": str})
    return df


# Este bloco só roda quando executamos ESTE arquivo diretamente.
# Quando o main.py importa a função, ele é ignorado.
if __name__ == "__main__":
    df = carregar_dados()
    print(f"{len(df)} registros carregados")
    print(df.head())
```

Conceito

Pense no **DataFrame** como uma planilha do Excel dentro do Python: tem colunas com nome e linhas numeradas a partir de 0. Para pegar uma coluna, usamos `df["nome_da_coluna"]`.

Entendendo o código

- `import pandas as pd` traz a biblioteca pandas. O apelido `pd` é uma convenção usada no mundo todo.
- `pd.read_csv(...)` lê o arquivo e devolve um **DataFrame**, que é uma tabela com linhas e colunas.
- `dtype={"zona": str, "secao": str}` manda ler essas colunas como **texto**. Sem isso, `0001` viraria o número `1`.
- `return df` devolve a tabela para quem chamou a função.
- `if __name__ == "__main__":` faz o bloco de baixo rodar **só** quando executamos este arquivo diretamente. Quando outro arquivo importa a função, esse bloco é ignorado.
- `df.head()` mostra as 5 primeiras linhas. O pandas esconde algumas colunas com `...` para caber na tela.

Rodando a etapa

DIGITE NO TERMINAL (POWERSHELL)

```
python etapa1_carregar.py
```

RESULTADO ESPERADO NO TERMINAL

```
32 registros carregados
    ano estado    cidade zona  ...      candidato partido numero votos
0  2024     SP  Campinas  101  ...    Ana Ribeiro     PFA     11   412
1  2024     SP  Campinas  101  ...  Bruno Tavares     PNB     22   388
2  2024     SP  Campinas  101  ...   Carla Mendes     PVC     33   157
3  2024     SP  Campinas  101  ...    Ana Ribeiro     PFA     11   365
4  2024     SP  Campinas  101  ...  Bruno Tavares     PNB     22   401

[5 rows x 10 columns]
```

Desafio para os alunos

1. Mostre as **5 últimas** linhas usando `df.tail()`.
2. Mostre o nome de todas as colunas com `print(df.columns)`.
3. Mostre só a coluna de candidatos: `print(df["candidato"])`.

Etapa 2 · Resumo geral

Objetivo: Mostrar um quadro com os totais: registros, votos, cidades e candidatos.

Crie o arquivo `etapa2_resumo.py` na pasta do projeto:

ARQUIVO: ETAPA2_RESUMO.PY

```
"""Etapa 2: mostrar um quadro com os totais gerais."""

from rich.panel import Panel

from etapa1_carregar import carregar_dados
from formatacao import console, numero_br


def mostrar_resumo(df):
    total_registros = len(df)                    # quantidade de linhas
    total_votos = df["votos"].sum()              # soma da coluna votos
    total_cidades = df["cidade"].nunique()       # quantas cidades diferentes
    total_candidatos = df["candidato"].nunique() # quantos candidatos diferentes

    # [bold]...[/] é a forma do rich deixar o texto em negrito
    texto = (
        f"[bold]Registros:[/] {numero_br(total_registros)}    "
        f"[bold]Votos:[/] {numero_br(total_votos)}    "
        f"[bold]Cidades:[/] {total_cidades}    "
        f"[bold]Candidatos:[/] {total_candidatos}"
    )

    # Panel desenha uma caixa com título em volta do texto
    console.print(Panel(texto, title="Eleições 2024 (dados fictícios)", expand=False))


if __name__ == "__main__":
    df = carregar_dados()
    mostrar_resumo(df)
```

Entendendo o código

- `from etapa1_carregar import carregar_dados` **reaproveita** a função da etapa 1. Não precisamos escrevê-la de novo.
- `len(df)` conta quantas linhas a tabela tem.
- `df["votos"].sum()` soma todos os valores da coluna votos.
- `df["cidade"].nunique()` conta quantos valores **diferentes** existem na coluna (cidades repetidas contam uma vez só).
- `[bold]...[/]` é a forma do rich deixar um trecho em **negrito**.
- `Panel(...)` desenha uma caixa com título em volta do texto.

Rodando a etapa

DIGITE NO TERMINAL (POWERSHELL)

```
python etapa2_resumo.py
```

RESULTADO ESPERADO NO TERMINAL

```
──────────── Eleições 2024 (dados fictícios) ────────────
 Registros: 32    Votos: 9.100    Cidades: 5    Candidatos: 16
─────────────────────────────────────────────────────────
```

Desafio para os alunos

Acrescente ao quadro o **total de partidos**, usando `df["partido"].nunique()`.

Etapa 3 · Amostra dos dados

Objetivo: Mostrar as primeiras linhas do CSV em uma tabela bonita, só com as colunas mais importantes.

Crie o arquivo `etapa3_amostra.py` na pasta do projeto:

ARQUIVO: ETAPA3_AMOSTRA.PY

```
"""Etapa 3: mostrar as primeiras linhas dos dados em uma tabela."""

from rich.table import Table

from etapa1_carregar import carregar_dados
from formatacao import console


def mostrar_amostra(df, linhas=5):
    # Escolhemos só algumas colunas para a tabela caber na tela
    colunas = ["estado", "cidade", "secao", "cargo", "candidato", "partido", "votos"]

    # 1. Cria a tabela vazia
    tabela = Table(title=f"Primeiras {linhas} linhas", header_style="bold cyan")

    # 2. Adiciona o cabeçalho (uma coluna por vez)
    for coluna in colunas:
        tabela.add_column(coluna)

    # 3. Adiciona as linhas. head() pega as primeiras linhas do DataFrame
    for _, linha in df.head(linhas).iterrows():
        # A tabela só aceita texto, por isso o str()
        valores = [str(linha[coluna]) for coluna in colunas]
        tabela.add_row(*valores)

    # 4. Mostra no terminal
    console.print(tabela)


if __name__ == "__main__":
    df = carregar_dados()
    mostrar_amostra(df)
```

Entendendo o código

- `linhas=5` é um **valor padrão**: se ninguém disser quantas linhas, a função usa 5.
- Montar uma tabela no rich tem sempre 4 passos: **criar** (`Table`), **cabeçalho** (`add_column`), **linhas** (`add_row`) e **mostrar** (`console.print`).
- `df.head(linhas).iterrows()` percorre as primeiras linhas, uma de cada vez. O `_` guarda o número da linha, que não vamos usar.
- `[str(linha[coluna]) for coluna in colunas]` monta uma **lista** com os valores da linha, convertidos para texto.
- O `*` em `add_row(*valores)` "espalha" a lista: cada item vira um argumento separado.

Rodando a etapa

DIGITE NO TERMINAL (POWERSHELL)

```
python etapa3_amostra.py
```

RESULTADO ESPERADO NO TERMINAL

```
                        Primeiras 5 linhas
```

estado	cidade	secao	cargo	candidato	partido	votos
SP	Campinas	0001	Prefeito	Ana Ribeiro	PFA	412
SP	Campinas	0001	Prefeito	Bruno Tavares	PNB	388
SP	Campinas	0001	Prefeito	Carla Mendes	PVC	157
SP	Campinas	0002	Prefeito	Ana Ribeiro	PFA	365
SP	Campinas	0002	Prefeito	Bruno Tavares	PNB	401

Desafio para os alunos

1. Mostre **10 linhas** em vez de 5 (mude a chamada para `mostrar_amostra(df, 10)`).
2. Inclua a coluna `zona` na lista `colunas`.

Etapa 4 · Votos por partido

Objetivo: Somar os votos de cada partido, calcular o percentual e mostrar uma barra para comparar.

Crie o arquivo `etapa4_partidos.py` na pasta do projeto:

ARQUIVO: ETAPA4_PARTIDOS.PY

```
"""Etapa 4: somar os votos de cada partido."""

from rich.table import Table

from etapa1_carregar import carregar_dados
from formatacao import console, numero_br


def calcular_votos_por_partido(df):
    # groupby("partido") separa as linhas por partido;
    # ["votos"].sum() soma os votos de cada grupo.
    por_partido = df.groupby("partido")["votos"].sum()

    # Ordena do partido mais votado para o menos votado
    por_partido = por_partido.sort_values(ascending=False)
    return por_partido


def mostrar_votos_por_partido(por_partido):
    total = por_partido.sum()

    tabela = Table(title="Votos por partido", header_style="bold cyan")
    tabela.add_column("Partido")
    tabela.add_column("Votos", justify="right")  # números alinhados à direita
    tabela.add_column("%", justify="right")
    tabela.add_column("Gráfico")

    # items() devolve pares (partido, votos)
    for partido, votos in por_partido.items():
        percentual = votos / total * 100

        # Barra de texto: um bloco "█" a cada 2%
        barra = "█" * round(percentual / 2)

        tabela.add_row(
            partido,
            numero_br(votos),
            numero_br(percentual, 1),
            f"[green]{barra}[/]",
        )

    console.print(tabela)


if __name__ == "__main__":
    df = carregar_dados()
    por_partido = calcular_votos_por_partido(df)
    mostrar_votos_por_partido(por_partido)
```

Conceito

groupby é uma das ferramentas mais usadas em análise de dados. A ideia é sempre: **separar** em grupos, **calcular** algo em cada grupo (soma, média, contagem) e **juntar** os resultados.

Entendendo o código

- `df.groupby("partido")` separa as linhas em **grupos**, um para cada partido.
- `["votos"].sum()` soma os votos **dentro de cada grupo**. O resultado tem uma linha por partido.
- `sort_values(ascending=False)` ordena do maior para o menor.
- A etapa tem **duas funções**: uma que **calcula** e outra que **mostra**. Assim podemos usar o cálculo em outro lugar sem a tabela.
- `por_partido.items()` devolve pares `(partido, votos)` para o `for`.
- `"█" * 20` repete o caractere 20 vezes. Usamos um bloco a cada 2% para desenhar a barra.

Rodando a etapa

DIGITE NO TERMINAL (POWERSHELL)

```
python etapa4_partidos.py
```

RESULTADO ESPERADO NO TERMINAL

```
                Votos por partido
┌─────────┬─────────┬──────┬──────────────────────┐
│ Partido │   Votos │    % │ Gráfico              │
├─────────┼─────────┼──────┼──────────────────────┤
│ PFA     │   3.603 │ 39,6 │ ████████████████████ │
│ PVC     │   2.915 │ 32,0 │ ████████████████     │
│ PNB     │   2.582 │ 28,4 │ ██████████████       │
└─────────┴─────────┴──────┴──────────────────────┘
```

Desafio para os alunos

Crie uma tabela de **votos por estado**. Dica: copie as duas funções e troque `"partido"` por `"estado"`.

Etapa 5 · Prefeitos eleitos

Objetivo: Descobrir o candidato a prefeito mais votado em cada cidade e o percentual de votos dele.

Crie o arquivo `etapa5_vencedores.py` na pasta do projeto:

ARQUIVO: ETAPA5_VENCEDORES.PY

```
"""Etapa 5: descobrir o prefeito eleito em cada cidade."""

from rich.table import Table

from etapa1_carregar import carregar_dados
from formatacao import console, numero_br


def calcular_vencedores(df):
    # 1. Fica só com as linhas de prefeito (ignora vereadores)
    prefeitos = df[df["cargo"] == "Prefeito"]

    # 2. Soma os votos de cada candidato, juntando todas as seções da cidade
    por_candidato = prefeitos.groupby(
        ["estado", "cidade", "candidato", "partido"], as_index=False
    )["votos"].sum()

    # 3. Calcula o percentual de cada candidato dentro da sua cidade.
    #    transform("sum") coloca, em cada linha, o total de votos da cidade.
    total_da_cidade = por_candidato.groupby("cidade")["votos"].transform("sum")
    por_candidato["percentual"] = (por_candidato["votos"] / total_da_cidade * 100).round(1)

    # 4. Ordena do mais votado para o menos votado
    #    e fica com o primeiro de cada cidade (o vencedor)
    ordenado = por_candidato.sort_values("votos", ascending=False)
    vencedores = ordenado.drop_duplicates("cidade")

    # 5. Ordena por nome da cidade para ficar mais fácil de ler
    return vencedores.sort_values("cidade")


def mostrar_vencedores(vencedores):
    tabela = Table(title="Prefeitos eleitos", header_style="bold cyan")
    tabela.add_column("UF")
    tabela.add_column("Cidade")
    tabela.add_column("Candidato", style="bold")
    tabela.add_column("Partido")
    tabela.add_column("Votos", justify="right")
    tabela.add_column("%", justify="right")

    for _, linha in vencedores.iterrows():
        tabela.add_row(
            linha["estado"],
            linha["cidade"],
            linha["candidato"],
            linha["partido"],
            numero_br(linha["votos"]),
            numero_br(linha["percentual"], 1),
        )

    console.print(tabela)


if __name__ == "__main__":
    df = carregar_dados()
    vencedores = calcular_vencedores(df)
    mostrar_vencedores(vencedores)
```

Entendendo o código

- `df[df["cargo"] == "Prefeito"]` é um **filtro**: fica só com as linhas em que o cargo é Prefeito.
- O `groupby` com **várias colunas** soma os votos de cada candidato juntando todas as seções. `as_index=False` mantém essas colunas como colunas normais da tabela.
- `transform("sum")` calcula o total de votos da cidade e repete esse valor em **cada linha** da cidade. Assim dá para dividir os votos do candidato pelo total da cidade dele.
- `round(1)` arredonda para uma casa decimal.
- Para achar o vencedor, ordenamos do mais votado para o menos votado e usamos `drop_duplicates("cidade")`, que fica só com a **primeira** linha de cada cidade.

Rodando a etapa

DIGITE NO TERMINAL (POWERSHELL)

```
python etapa5_vencedores.py
```

RESULTADO ESPERADO NO TERMINAL

```
                    Prefeitos eleitos
```

UF	Cidade	Candidato	Partido	Votos	%
SP	Campinas	Bruno Tavares	PNB	789	41,6
PE	Olinda	Iara Santos	PNB	568	51,1
PE	Recife	Fabio Lins	PVC	1.008	41,9
SP	Santos	Elisa Moura	PNB	668	52,1
MG	Uberlandia	Karina Alves	PFA	843	50,6

Desafio para os alunos

Descubra o **vereador** mais votado de cada cidade. Dica: troque `"Prefeito"` por `"Vereador"` no filtro. Por que só aparecem duas cidades? (Olhe o CSV.)

Etapa 6 · Salvar o resultado

Objetivo: Gravar a tabela de prefeitos eleitos em um novo arquivo CSV, pronto para abrir no Excel.

Crie o arquivo `etapa6_exportar.py` na pasta do projeto:

ARQUIVO: ETAPA6_EXPORTAR.PY

```
"""Etapa 6: salvar o resultado em um novo arquivo CSV."""

from etapa1_carregar import carregar_dados
from etapa5_vencedores import calcular_vencedores
from formatacao import console

CAMINHO_RESULTADO = "dados/resultado_prefeitos.csv"


def salvar_csv(tabela):
    # to_csv grava a tabela em um arquivo. Os parâmetros deixam o
    # arquivo pronto para abrir no Excel em português:
    #   index=False         -> não grava a numeração das linhas
    #   sep=";"             -> separa as colunas com ponto e vírgula
    #   decimal=","         -> usa vírgula nas casas decimais (41,6)
    #   encoding="utf-8-sig" -> mantém os acentos corretos no Excel
    tabela.to_csv(
        CAMINHO_RESULTADO,
        index=False,
        sep=";",
        decimal=",",
        encoding="utf-8-sig",
    )
    console.print(f"Resultado salvo em: {CAMINHO_RESULTADO}")


if __name__ == "__main__":
    df = carregar_dados()
    vencedores = calcular_vencedores(df)
    salvar_csv(vencedores)
```

Entendendo o código

- Esta etapa usa o cálculo da etapa 5 (`calcular_vencedores`) para ter o que salvar.
- `to_csv` faz o caminho contrário do `read_csv`: transforma a tabela em arquivo.
- O Excel em português espera **ponto e vírgula** entre colunas e **vírgula** nos decimais. Por isso usamos `sep=";"` e `decimal=","`.
- `encoding="utf-8-sig"` garante que os acentos apareçam certos quando o Excel abrir o arquivo.

Rodando a etapa

DIGITE NO TERMINAL (POWERSHELL)

```
python etapa6_exportar.py
```

RESULTADO ESPERADO NO TERMINAL

```
Resultado salvo em: dados/resultado_prefeitos.csv
```

Abra o arquivo `dados/resultado_prefeitos.csv` criado na pasta `dados`. O conteúdo deve ser:

ARQUIVO: DADOS/RESULTADO_PREFEITOS.CSV

```
estado;cidade;candidato;partido;votos;percentual
SP;Campinas;Bruno Tavares;PNB;789;41,6
PE;Olinda;Iara Santos;PNB;568;51,1
PE;Recife;Fabio Lins;PVC;1008;41,9
SP;Santos;Elisa Moura;PNB;668;52,1
MG;Uberlandia;Karina Alves;PFA;843;50,6
```

Desafio para os alunos

Salve também os **votos por partido** em `dados/votos_partido.csv`. Dica: importe `calcular_votos_por_partido` da etapa 4 e use `to_csv` no resultado.

Parte 5 · Juntando tudo

O main.py chama todas as etapas, uma depois da outra.

Agora que todas as etapas funcionam sozinhas, o `main.py` só precisa **importar** as funções e chamá-las em ordem. Repare que ele não tem nenhum cálculo: todo o trabalho está nas etapas.

ARQUIVO: MAIN.PY

```
"""Programa principal: executa todas as etapas, uma depois da outra."""

from etapa1_carregar import carregar_dados
from etapa2_resumo import mostrar_resumo
from etapa3_amostra import mostrar_amostra
from etapa4_partidos import calcular_votos_por_partido, mostrar_votos_por_partido
from etapa5_vencedores import calcular_vencedores, mostrar_vencedores
from etapa6_exportar import salvar_csv

# Etapa 1: ler os dados
df = carregar_dados()

# Etapa 2: totais gerais
mostrar_resumo(df)

# Etapa 3: primeiras linhas
mostrar_amostra(df)

# Etapa 4: votos por partido
por_partido = calcular_votos_por_partido(df)
mostrar_votos_por_partido(por_partido)

# Etapa 5: prefeitos eleitos
vencedores = calcular_vencedores(df)
mostrar_vencedores(vencedores)

# Etapa 6: salvar o resultado
salvar_csv(vencedores)
```

Rodando o programa completo

DIGITE NO TERMINAL (POWERSHELL)

```
python main.py
```

O resultado é o programa completo, igual ao mostrado no começo do tutorial, na seção **O que vamos construir**.

Confira a estrutura da pasta

No final, a pasta do projeto deve ter estes arquivos:

PASTA DO PROJETO

```
eleicoes/
├── .venv/                    ← ambiente virtual (criado pelo Python)
├── dados/
│   ├── eleicoes.csv          ← dados de entrada
│   └── resultado_prefeitos.csv ← criado pela etapa 6
├── requirements.txt          ← lista de bibliotecas
├── formatacao.py             ← funções de apoio
├── etapa1_carregar.py
├── etapa2_resumo.py
├── etapa3_amostra.py
├── etapa4_partidos.py
├── etapa5_vencedores.py
├── etapa6_exportar.py
└── main.py                   ← roda todas as etapas
```

Por que separar em etapas?

Dividir um programa em **funções** e **arquivos** pequenos deixa o código mais fácil de entender, testar e mudar. Se amanhã quisermos mostrar um gráfico em vez de uma tabela, mexemos só em uma etapa, e as outras continuam iguais.

Parte 6 · Problemas comuns e como resolver

Mensagem de erro	Causa provável	Como resolver
`Python was not found` ou abre a Microsoft Store	Python não instalado, ou terminal aberto antes da instalação	Feche e abra o terminal. Se continuar, reinstale marcando **Add python.exe to PATH**.
"a execução de scripts foi desabilitada"	O Windows bloqueia o `Activate.ps1`	Rode `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` e ative de novo.
`ModuleNotFoundError: No module named 'pandas'` (ou `'rich'`)	Ambiente virtual não ativado ou bibliotecas não instaladas	Ative o `.venv`, rode `pip install -r requirements.txt` e selecione o interpretador `.venv` no VS Code.
`ModuleNotFoundError: No module named 'etapa1_carregar'`	Nome do arquivo diferente ou arquivo em outra pasta	Confira se o arquivo existe com o nome exato e está na mesma pasta das outras etapas.
`FileNotFoundError: ... dados/eleicoes.csv`	O VS Code abriu outra pasta, ou o CSV não está em `dados`	Abra a **pasta do projeto** em Arquivo → Abrir Pasta. Confira o caminho `dados/eleicoes.csv`.
`KeyError: 'votos'`	Nome de coluna digitado errado (no código ou no CSV)	Compare o nome no código com a primeira linha do CSV. Maiúsculas e acentos fazem diferença.
`IndentationError`	Espaços no começo da linha fora do lugar	Use 4 espaços para cada nível. O código dentro de `def`, `for` e `if` fica mais à direita.

Parte 7 · Resumo dos comandos

Comando	O que faz
`python --version`	Confere se o Python está instalado
`python -m venv .venv`	Cria o ambiente virtual (uma vez por projeto)
`.\.venv\Scripts\Activate.ps1`	Ativa o ambiente virtual (cada vez que abrir um terminal novo)
`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`	Libera a ativação do venv no Windows (uma vez por computador)
`pip install -r requirements.txt`	Instala as bibliotecas do projeto
`python etapa1_carregar.py`	Roda uma etapa sozinha
`python main.py`	Roda o programa completo
`deactivate`	Desativa o ambiente virtual